Efficient Conversational Memory Retrieval in AI Agents Using Hybrid Search and NVMe Storage

1 Topic

This research focuses on improving how AI agents find information from long conversations. AI agents may need to remember names, dates, technical details, and information discussed many days or months earlier.

Finding the correct information becomes more difficult as the amount of stored memory increases. Keyword search can find exact words, while semantic search can find information with similar meaning. Combining these methods may improve retrieval.

The project will also study how storage type affects retrieval speed when the memory database becomes larger. In particular, I will compare memory retrieval using RAM and NVMe storage.

2 Research Question

How can question-type-aware hybrid search and NVMe storage improve the accuracy and performance of conversational memory retrieval as memory size and concurrent requests increase?

3 Hypothesis

I expect that combining keyword and semantic search will retrieve relevant memories more accurately than using either method alone.

I also expect that an adaptive search method will perform better for different types of questions. NVMe storage may help maintain good retrieval performance when the memory database becomes too large to keep entirely in RAM.

The hypothesis will be challenged if hybrid search does not improve retrieval accuracy, or if NVMe storage provides no meaningful performance benefit in the tested workloads.

4 Related Work

The following studies provide background for conversational memory, memory retrieval, and search methods.

Study	What it measured or evaluated	Setup
MemGPT / Letta (2023)	Memory retrieval accuracy, ROUGE-L, document question answering and nested key-value retrieval.	Used GPT-3.5 Turbo, GPT-4 and GPT-4 Turbo. Tested multi-session conversations and synthetic nested key-value data containing 140 pairs, with up to four lookup levels. Used hierarchical memory with main context and external storage.
LongMemEval (2025)	Memory recall accuracy, question-answering accuracy, temporal reasoning and knowledge updates.	500 questions across five main categories, with approximately 115K tokens per history in the LongMemEval-S version. Tested GPT-4o, Llama 3.1 70B and Llama 3.1 8B. Used Stella V5 1.5B for dense retrieval and Llama 3.1 8B for memory indexing.
Zep / Graphiti (2025)	Retrieval accuracy, temporal reasoning, response latency and context-token usage.	DMR: 500 conversations with five sessions each. LongMemEval: approximately 115K tokens per history. Used BGE-M3 for embeddings and reranking, GPT-4o-mini for graph construction, and GPT-4o / GPT-4o-mini for answering. Retrieved top-20 facts and entity nodes.
Mem0 (2025)	Answer accuracy, retrieval latency, token consumption and memory efficiency.	LoCoMo: 10 conversations, approximately 26,000 tokens each. Used GPT-4o-mini, text-embedding-3-small and Neo4j for graph memory. Compared memory retrieval with RAG using chunk sizes of 128–8192 tokens.
A-MEM (2025)	F1, BLEU-1, ROUGE, METEOR, SBERT similarity and token usage.	LoCoMo (7,512 QA pairs) and DialSim (1,300 sessions). Tested GPT-4o, GPT-4o-mini, Qwen 2.5 (1.5B/3B) and Llama 3.2 (1B/3B). Used all-MiniLM-L6-v2 embeddings and top-k retrieval, mainly k=10. Llama 3.2 1B local inference tested on a single GPU
BM25 Robertson and Zaragoza (2009)	BM25 ranking model, probabilistic relevance, term weighting, and parameter optimisation.	A theoretical review of probabilistic information retrieval, based on previous retrieval studies. It does not report one specific experimental dataset or hardware configuration.

5 Method

I plan to build a small conversational memory retrieval system and compare different search methods.

1. Dataset Preparation

I will use publicly available conversational memory datasets, mainly LoCoMo and LongMemEval.

The dataset will contain conversation histories and questions with known relevant information. I will prepare a manageable subset for the experiments.

Questions will include:

- Exact information, such as names, dates, and numbers.
- Semantic questions that use different words from the original conversation.
- Questions about people, events, or technical details.
- Questions that require information from multiple conversations.

2. Search Methods

I will compare four retrieval methods:

1. **Keyword search:** BM25 will find memories based on matching words.
2. **Semantic search:** Sentence Transformers will find memories with similar meanings.
3. **Fixed hybrid search:** Keyword and semantic search results will be combined using fixed weights.
4. **Adaptive hybrid search:** The system will identify the question type and adjust the keyword and semantic search weights accordingly.

For example, a question asking for an exact date may benefit more from keyword search. A question asking about the meaning of an earlier discussion may benefit more from semantic search.

3. Storage Experiments

I will test the retrieval system using different storage configurations:

- RAM-based storage.
- SSD-based storage, if available.
- NVMe-based storage.

I will gradually increase the memory database size and test different numbers of simultaneous requests.

The main purpose is to understand how storage affects retrieval speed and system performance as the workload increases.

6 Tools and Technologies

The project will mainly use:

- **Python** for implementation and experiments.
- **BM25** for keyword search.
- **Sentence Transformers** for semantic search.
- **FAISS** for vector similarity search.
- **Linux** for running experiments.
- **Pandas and Matplotlib** for analysing and presenting results.

7 Evaluation and Measurements

I will use the following metrics:

- **Recall@5 and Recall@10:** Whether relevant memories appear in the top results.
- **Precision@5:** How many of the top five results are relevant.
- **MRR:** How highly the first relevant result is ranked.
- **Retrieval latency:** How long the system takes to retrieve memories.
- **Throughput:** How many requests the system can handle per second.
- **Storage usage:** How much space is needed for the memory database and search indexes.

I will compare the results across search methods, database sizes, storage types, and concurrent request levels.

8 Work Plan Until May 2027

Period	Planned work
November 2026	Read related papers and finalise the research design.
December 2026	Prepare the dataset and implement BM25 keyword search.
January 2027	Implement semantic search and fixed hybrid search.
February 2027	Develop adaptive hybrid search and prepare storage experiments.
March 2027	Run experiments with different database sizes and storage types.
April 2027	Analyse results, compare methods, and repeat important experiments.
May 2027	Complete the report and prepare the final presentation.

9 Resources

- **GPU:** NVIDIA RTX 3060 (12 GB) or similar. Estimated 10–20 GPU hours for embedding generation and experiments.
- **CPU and RAM:** A modern multi-core CPU with 16–32 GB RAM.
- **Storage:** 20–50 GB SSD/NVMe storage for conversation datasets, vector indexes, and experimental results.
- **Storage types:** RAM-based storage and SSD/NVMe storage for performance comparison.
- **Software:** Python, Linux, FAISS, BM25, Sentence Transformers, Pandas, and Matplotlib.
- **API budget:** Approximately €0–30. Local embedding models will be used to reduce API costs. Paid API usage will be optional.

10 Risks and Fallback

- **Limited GPU resources:** Use smaller embedding models and CPU-based processing.
- **Large dataset processing:** Start with a smaller subset of LoCoMo and LongMemEval.
- **Storage limitations:** Reduce database size or use available SSD/NVMe storage.
- **Slow experiments:** Reduce the number of concurrent requests and repeat only important tests.
- **API cost:** Prefer open-source models running locally instead of paid APIs.
- **Limited time:** Focus on comparing BM25, semantic search, and hybrid search first. Treat adaptive search and larger storage experiments as extensions if time allows.